KavAI™

Single-Agent ADK Data Query Agent

Joan Vendrell Gallart

1. Agent Architecture

We consider a tool-using agent designed to answer filtering queries over a structured sensor dataset. The agent receives a natural-language user request, extracts the required filtering conditions, and executes them through a restricted tool interface which exposes a single operation, `filter_records_adk`, which applies exactly one filtering criterion per call. Therefore, multi-condition queries are handled through sequential tool composition: the output of one tool call becomes the input to the next.

The architecture has four main components. First, the *User Query Parser* maps the user request into a structured list of admissible filters. The supported filters are limited to location, sensor type, value range, and anomaly label. Second, the *Controller* decides the execution order of the filters and ensures that each tool call contains only one criterion. Third, the *Tool Executor* calls `filter_records_adk` and passes intermediate results between calls when the query contains multiple conditions. Finally, the *Response Generator* (based on GPT) converts the final filtered records into a concise answer for the user.

This design enforces correctness by constraining the agent to a small action space and prevents unsupported behavior by rejecting queries outside the allowed filter set. Efficiency is controlled by limiting the number of tool calls to the number of valid filtering conditions in the request.

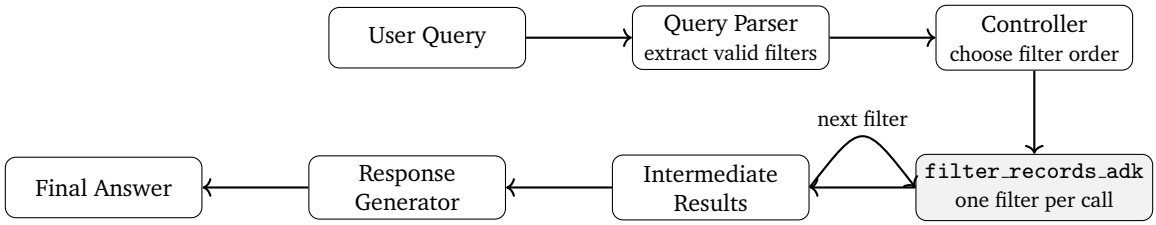

Figure 1: Agent architecture for constrained sensor dataset filtering. The controller decomposes multi-condition requests into sequential one-filter tool calls.

2. Tool Design

The agent is equipped with a single external tool, `filter_records_adk`, which performs deterministic filtering over the uploaded sensor dataset. The tool is intentionally narrow: each call applies exactly one filtering criterion. This restriction simplifies verification, prevents ambiguous tool usage, and makes multi-step reasoning explicit. The tool accepts one of four admissible filter types: `location`, `sensor_type`, `value_range`, or `anomaly_label`. For single-condition requests, the agent calls the tool once. For multi-condition requests, the agent calls the tool sequentially, passing the previous output as `previous_results`. Thus, conjunctions of filters are implemented as a pipeline of simple deterministic operations.

Algorithm 1 Sequential Tool-Based Filtering

Require: User query q, sensor dataset $\mathcal{D}$

Extract valid filters $\mathcal{F} = \{f_1, \ldots, f_k\}$ from q ▷ LLM-based
$\mathcal{R} \leftarrow \mathcal{D}$
for $i = 1$ to k **do**
    $\mathcal{R} \leftarrow$ `filter_records_adk`$(f_i, \mathcal{R})$
end for
return $\mathcal{R}$

3. Example Queries and Outputs

We tested the agent on both valid and invalid user queries. Valid queries are those that can be answered using only the supported filters: location, sensor type, value range, and anomaly label. Invalid queries are declined when they require unsupported operations such as counting, averaging, aggregation, or general reasoning outside dataset filtering.

```
[user]:  Show gas readings in Area A
[sensor_filter_agent]:  The filtered results for gas readings in Area A are as follows:
1.  Asset ID: A-101
```

```
   - Timestamp:  2026-04-01 10:00:00
   - Value:  45.0
   - Anomaly Label:  False
2.  Asset ID: A-113
   - Timestamp:  2026-04-03 08:20:00
   - Value:  55.0
   - Anomaly Label:  True
3.  Asset ID: A-117
   - Timestamp:  2026-04-03 12:30:00
   - Value:  65.0
   - Anomaly Label:  True
4.  Asset ID: A-125
   - Timestamp:  2026-04-04 13:30:00
   - Value:  35.0
   - Anomaly Label:  False
```

This query is valid because it can be decomposed into two supported filters: `sensor_type = gas` and `location = Area A`. The agent therefore executes the filtering pipeline and returns the matching records.

```
[user]:  What is the average temperature?
[sensor_filter_agent]:  I'm sorry, but I can only answer filtering questions about location, sensor
type, value range, and anomaly label.  Please let me know if you have any filtering requests!
```

This query is invalid because it requires an aggregation operation, namely computing an average. Since aggregation is outside the supported tool interface, the agent correctly declines the request instead of producing an unsupported answer.

4. Failure Cases

The agent is reliable for supported filtering queries, but it is intentionally limited by its tool interface. The main failure mode is *unsupported computation* where, by design, the agent should therefore decline these requests. Other failures come from ambiguous or incorrect parsing. Similarly, "not anomalies" must be mapped correctly to `anomaly_label=False`; otherwise, the tool may execute but return incorrect records.

Failure	Example	Behavior
Aggregation	"What is the average temperature?"	Decline.
Counting	"How many anomalies are in Area B?"	Decline.
Ambiguity	"Show high gas readings."	Clarify/decline.
Parsing error	"Show readings that are not anomalies."	Validate mapping.
Unsupported task	"Sort sensors by value."	Decline.

5. Execution Control Observations

We tested the agent with different execution limits. With `max_iterations=1`, the agent behaved as a single-shot system. This was efficient for simple queries such as *"Show gas readings in Area A,"* but it was not sufficient for multi-condition queries such as *"Show gas anomalies in Zone 2 with value above 60."* This query requires several filters: `sensor_type=gas`, `location=Zone 2`, `anomaly_label=True`, and `value > 60`. Since the tool can apply only one filter per call, a single iteration cannot complete the full filtering chain. Unlike previous case, with `max_iterations=5`, the agent could decompose complex requests into multiple sequential tool calls.

Overall, `max_iterations=5` with low temperature gave the best behavior: it preserved controlled execution while allowing enough steps for valid multi-condition filtering, despite slightly increasing computational time and cost.

6. What You Would Improve Next

Given more time, I would improve three parts of the system. First, I would add verified tools for counting, averaging, sorting, and summarizing, so the agent could safely answer aggregation queries. Second, I would strengthen query validation, especially for ambiguous phrases such as "high readings" and negations such as "not anomalies." Third, I would automate execution control by selecting `max_iterations` from the estimated query complexity. These changes would expand the agent's capabilities while keeping its behavior constrained, interpretable, and safe. In addition, I would implement RL framework, see *rldesign.pdf*, for a dynamic adjustment of model's policy increasing the accuracy.